

Class06: R Functions

Sofia Jaravata (A19160915)

Table of contents

Background	1
Q1. A first function	1
Q2. A <code>generate_dna()</code> function	2
Q3. Write a <code>generate_protein()</code> function	5
Q4. Generate random protein sequences of length 6 to 13	6
Q5. BLASTp search against nr — are your peptides “unique in nature”?	7
Q6. Connecting your findings to immunology (MHC class II and T-cell activation)	7

Background

All functions in R have at least 3 things: - a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separate inputs that go inside the brackets when we call the function), - the *body* (the lines of R code that do the work of the function).

Q1. A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “required” or “optional”. The later have fall-back default values that will be used if the user does not specify them. > Q1: a. Your first version of the function should add two input numbers together. For example, `add(4,7)` should return 11. [1 pt]

```
add <- function(x,y){ # defines an add function for two inputs,
#doing this so I can add two values, x and y
  x+y ##returns sum of x+y
}
```

Can we use our new function?

```
add(4,7) #add two values together
```

```
[1] 11
```

Q2: b. For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7,10))`. [1 pt] Q2: c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the... (dots) argument or the base R function `sum()`. [2 pts]

```
add <- function(...){ # function takes in any amount of arguments,
  #doing this so I can put in any amount of numbers
  #and get the sum of all of them
  sum(...) # returns the sum of all the values
}
```

```
b <- add(sum(c(4,7,10)))
c <- add(sum(c(1,2,3,-4)))

b
```

```
[1] 21
```

```
c
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

Q2. A `generate_dna()` function

A useful function here is the “base R” `sample()` function.

```
# "1:5" chooses from numbers 1-5,
#"size 7" is how many values you want to be printed out,
#"replace = TRUE" allows for numbers to repeat;
#doing these to generate a list of 7 numbers 1-5 with repeats
sample(1:5, size = 7, replace = TRUE)
```

```
[1] 2 5 1 2 5 3 3
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
# random choosing from nucleotides A,T,G,C,  
#print out 10 values from the nucleotides because size = 10,  
#allows for repeats because replace = TRUE  
sample(x = c("A","T","G","C"), size= 10, replace = TRUE, )
```

```
[1] "G" "T" "G" "G" "T" "A" "C" "C" "C" "T"
```

Q2: a. Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
# assigned generate_dna to a function with argument "len"  
#for a wanted sequence length,  
generate_dna <- function(len){  
  sample(x = c("A","T","G","C"), size=len, replace = TRUE, ) # sample() picks  
  #from A,T,G,C nucleotide values, with the size of however  
  #long the sequence length I want to print out,  
  #and allows for nucleotide values to repeat because replace = TRUE  
}  
  
generate_dna(10) # will generate a random nucleotide sequence with 10 letters
```

```
[1] "G" "C" "T" "G" "G" "A" "T" "A" "C" "T"
```

Q2: b. Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a *single character string* (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are `paste()`, `if()`, `cat()`

```

generate_dna <- function(len = 10, single.element = FALSE){ # 10 nucleotides
  #will be printed out because "len = 10"
  #and the data does not consist of one element
  #because "single.element = FALSE"
  ans <- sample(x = c("A","T","G","C"), size=len, replace = TRUE) # picks letters
  #from nucleotides A,T,G,C because sample() produces random letters,
  #the size is how long you want it to be when you ask with "generate_dna(#)"
  #because "size = len", and letters will be repeated because "replace = TRUE"

  if(single.element){ # checks whether "single.element" is TRUE; if TRUE,
    #then code within {} will run and if FALSE, code {} does not run;
    #doing this so it only runs when something is TRUE and not every time

    ans <- paste(ans, collapse = "") # "paste()" combines elements together,
    #ans is the function from above with producing random nucleotide sequence,
    #"collapse = "" " will combine everything with no separators
  }
  return(ans) # returns the sequence from the ans function
}

```

```

generate_dna(4) # will return sequence with 4 nucleotide letters

```

```

[1] "A" "A" "A" "A"

```

```

paste("A","C","G","T", collapse = "") # will paste ACGT without the

```

```

[1] "A C G T"

```

```

# "" quotations around each letter, just the numbers

```

- c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: >len9
CGAAGGCTG

```

generate_dna <- function(len = 10, single.element = FALSE){ #"len = 10" will be
  #how long the sequence will be with a default of 10, "single.element = FALSE"
  #means multiple elements will be in the response
  ans <- sample(x = c("A","T","G","C"), size=len, replace = TRUE) # takes from
  #a sample of A,T,G,C with the size equaling your desired length ("size = len")
  #, and letters with repeat because "replace = TRUE"

```

```

    if(single.element){ #if the function above within {} is true
      ans <- paste(ans, collapse = "") # paste(function) will paste the ans
      #function with no spaces in between because "collapse = " " "
    }
  }
  #return(ans)

  ### Format as FASTA with an ID line
  cat(paste(">len", len, "\n", sep = "")) # prints out in FASTA format
  cat(ans) # prints out the dna sequence
  cat("\n") # new line for separation between each ID

  return(ans) # returns the function ans
}

```

```

x <- generate_dna(9) # generates sequence of 9 letters

```

```

>len9
C G A T C T A A T

```

Q3. Write a generate_protein() function

Q3 Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return “WQRTAG”. Your function should:

- Use the single-letter code for all 20 standard amino acids and no other letters (see earlier list at the beginning of this handout) and include clear comments that explain each step of your function. [2 pts]

Example of the expected behavior generate_protein(6) # e.g. “WQRTAG”

```

# function with a default sequence length of 10
generate_protein <- function(len = 10) {

  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M",
         "F", "P", "S", "T", "W", "Y", "V") # 20 amino acid letters

  picks <- sample(aa, size = len, replace = TRUE) # randomly picks the amino
  #acids from above in "aa", with the size equaling the desired length because
  #"size = len", allowing for repeats because "replace = TRUE"

  paste(picks, collapse = "") # outputs the picks function above, and

```

```
#collapses any quotations between the amino acid letters
}
generate_protein(6) #generates 6 numbers from amino acids
```

```
[1] "FYCLTE"
```

Q4. Generate random protein sequences of length 6 to 13

Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp. For example:

```
for(l in 6:13){ #generate something of length "l"

  # make FASTA headers
  cat(">id.",l, "\n", sep = "")

  # combine header + sequence, print in FASTA format
  cat(generate_protein(l), "\n")
}
```

```
>id.6
QYTLLL
>id.7
GRKGKYN
>id.8
EYNENPLT
>id.9
NTYRMPNVN
>id.10
QGFQMSSKSC
>id.11
FIYVAKVSGRA
>id.12
WQKQWRTHASSQ
>id.13
DMLTARDHHDGNKS
```

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length(aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100%	100%	N
7	100%	100%	N
8	100%	100%	N
9	100%	100%	N
10	100%	80%	Y
11	90%	91%	Y
12	88.89%	75%	Y
13	75%	92%	Y

Q5. BLASTp search against nr — are your peptides “unique in nature”?

a. At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?
[1 pt]

At sequence length 10, my randomly generate peptides start to look “unique in nature”. This is because either the coverage or the identity hit (or both) do not equate to 100%.

b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe. [1 pt]

Short random peptides are often found in nr because nr has a large number of real protein sequences, and short sequences are small enough that they can be found in the huge collection of known proteins. As peptide length increases exponentially, the possible sequence space (20^L) become much larger than the amount of sequences displayed in the protein universe, thus are not present in nr because they are massive and most likely would not occur in a real protein.

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice

for the immune system to present very short peptides? [3 pt]

This minimum length is 10 because around length 10 in my Q5 data, the peptide sequences start to become unique in nr. Presenting very short peptides for the immune system would be a bad design choice because shorter peptides would be less specific, thus making it harder for the immune system to distinguish common peptides from foreign ones.